Table des matières

Labo 03 — Monter le projet dbt et le STAGING 1
Avant de commencer 2
Étape 1 — Le fichier de connexion, hors du dépôt 2
Étape 2 — Le projet dbt, à la racine du dépôt 3
Étape 3 — La macro des schémas 4
Étape 4 — Déclarer les sources 5
Étape 5 — Voir ce que dbt connaît 6
Étape 6 — La première vue de staging 6
Étape 7 — La vue qui retrouve les 25 483 commandes 8
Étape 8 — Relier deux modèles, et voir le graphe 9
Récapitulatif 10
Dépannage 11
Annexe — dbt sur Snowflake et sur Databricks 12

Labo 03 — Monter le projet dbt et le STAGING

Labo · Cours 3/9 — Formation Analytics Engineering

Ce que vous allez faire. Monter un projet dbt dans le dépôt créé au Cours 01, brancher sa connexion hors du dépôt, déclarer la couche `RAW` comme source, puis écrire vos premières vues de staging. La dernière retrouvera les 25 483 commandes de Boréal malgré l'extrait livré deux fois, et un modèle de contrôle reliera deux vues pour que le graphe ait sa première arête. Objectif de fin : `dbt debug` au vert, trois modèles construits, et un graphe qui remonte jusqu'aux sources.

Deux terminaux servent dans ce labo. L'Anaconda Prompt, environnement `formations` actif, sert à dbt et à Git. La ligne de commande DuckDB sert à vérifier ce que dbt a écrit, et elle doit être fermée quand dbt travaille.

La liste de contrôle du cours :

- ☐ Le schéma `prototype` du Cours 02 abandonné
- ☐ `profiles.yml` écrit hors du dépôt, avec un chemin absolu
- ☐ `dbt_project.yml` à la racine de `02_ae`, et `dbt debug` au vert

© 2026 iAlexMG. Tous droits réservés.

- ☐ La macro `generate_schema_name` en place, les schémas portent leur nom nu
- ☐ Les sources déclarées, et `dbt ls` qui les montre
- ☐ `stg_concessionnaires` : 156 lignes, 0 palier vide, 4 fiches fermées
- ☐ `stg_commandes` : 25 483 lignes, malgré les 26 538 de la source
- ☐ Le modèle de contrôle construit, et le graphe qui le relie aux deux vues

Le labo se fait en deux séances. La première va jusqu'à l'étape 5, la seconde porte les modèles de staging.

Avant de commencer

Ce labo suppose la couche `RAW` montée au Cours 01 et le dépôt `02_ae` créé à son étape 7. Il suppose aussi dbt installé, ce que le guide d'installation du parcours a fait.

Deux vérifications, dans l'Anaconda Prompt :

```
dbt --version
```

Vous devez lire `dbt=1.12.3` ou plus récent pour le cœur, et `duckdb: 1.11.0` sous `Plugins`. Si la ligne `Plugins` ne mentionne pas DuckDB, c'est l'adaptateur qui manque, et il s'installe par `pip install dbt-duckdb`.

Le Cours 02 vous a fait écrire des tables dans un schéma `prototype`, en vous annonçant qu'il servirait une fois. C'est maintenant. Dans la ligne de commande DuckDB, depuis `Projet_Boreal` :

```
drop schema if exists prototype cascade;
```

Ce que vous avez appris en l'écrivant reste ; les tables, elles, vont être refaites par dbt, mieux. Quittez ensuite DuckDB par `.exit` : un seul processus à la fois sur l'entrepôt, et dbt va l'ouvrir.

Étape 1 — Le fichier de connexion, hors du dépôt

Théorie : `Theorie_03_AE.md`, section 3.

dbt cherche `profiles.yml` dans un dossier `.dbt` de votre répertoire personnel. Créez-le, puis le fichier :

© 2026 iAlexMG. Tous droits réservés.

```
mkdir %USERPROFILE%\.dbt
notepad %USERPROFILE%\.dbt\profiles.yml
```

Sur macOS ou Linux, c'est `~/.dbt/profiles.yml`, avec l'éditeur de votre choix.

Contenu, en remplaçant le chemin par le vôtre :

```
boreal_ae:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: C:/Users/<vous>/.../Data_Domain/Projet_Boreal/plateforme/entrepot/boreal.duckdb
      schema: staging
      threads: 4
```

Quatre points méritent un mot.

Le nom `boreal_ae` en tête est le nom du profil, et le `dbt_project.yml` le réclamera à l'identique à l'étape suivante. Une faute de frappe ici produit une erreur qui parle de profil introuvable.

Le chemin est absolu, et les barres obliques sont celles de l'avant, même sous Windows. Un chemin relatif fonctionnerait tant que vous lancez dbt depuis le projet, et échouerait le jour où vous le lancez d'ailleurs.

`schema: staging` est le schéma par défaut de cette cible.

`threads: 4` autorise dbt à construire quatre modèles en parallèle quand le graphe le permet.

Piège — un `profiles.yml` vide est pire qu'absent. Sans fichier, dbt dit clairement qu'il n'a pas trouvé de profil et où il a cherché. Avec un fichier vide, dbt 1.12.3 s'arrête sur une trace Python et le message `cannot access local variable 'summary_message'`, qui ne dit rien à personne. Si vous créez le fichier maintenant pour le remplir plus tard, vous rencontrerez ce message.

Étape 2 — Le projet dbt, à la racine du dépôt

Théorie : `Theorie_03_AE.md`, section 2.

Vous avez déjà un dépôt, `02_ae`, créé au Cours 01. Le projet dbt va vivre à sa racine, et non dans un sous-dossier : le dépôt et le projet sont la même chose.

La commande `dbt init` existe, et c'est elle qu'on emploie quand on part de rien : elle crée un dossier et son squelette. Ici elle ne convient pas, pour deux raisons. Elle créerait un dossier dans votre dépôt, alors que vous voulez le projet à la racine. Et le nom d'un projet dbt ne peut pas commencer par un chiffre : `dbt init 02_ae` est refusé et vous laisse dans une invite qui redemande un nom, même avec l'option qui saute la configuration du profil.

© 2026 iAlexMG. Tous droits réservés.

Écrivez donc `dbt_project.yml` vous-même, à la racine de `02_ae` :

```yaml
name: boreal_ae
version: 1.0.0
profile: boreal_ae

model-paths: ["models"]
macro-paths: ["macros"]
test-paths: ["tests"]
target-path: "target"
clean-targets: ["target", "dbt_packages"]

models:
  boreal_ae:
    staging:
      +materialized: view
      +schema: staging
```

Remarquez la cascade du bloc `models:` : le nom du projet, puis le dossier, puis les réglages. Tout ce qui vivra dans `models/staging/` sera une vue, dans le schéma `staging`, sans avoir à le répéter.

Créez les dossiers, puis lancez la vérification :

```
mkdir models\staging
mkdir macros
dbt debug
```

Attendu, quatre lignes qui comptent :

```
Using profiles.yml file at C:\Users\<vous>\.dbt\profiles.yml
Using dbt_project.yml file at ...\Projet_Boreal\02_ae\dbt_project.yml
  profiles.yml file [OK found and valid]
  dbt_project.yml file [OK found and valid]
  Connection test: [OK connection ok]
All checks passed!
```

Si le test de connexion échoue sur un fichier introuvable, c'est le chemin du profil : recopiez-le depuis l'explorateur plutôt que de le retaper.

Étape 3 — La macro des schémas

Théorie : `Theorie_03_AE.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.

Avant d'écrire le moindre modèle, posez la macro qui décide où dbt écrit. Sans elle, vos vues atterriront dans un schéma nommé `staging_staging`, par concaténation du schéma du profil et de celui du dossier.

Dans `macros/generate_schema_name.sql` :

```
{% macro generate_schema_name(custom_schema_name, node) -%}

    {%- if custom_schema_name is none -%}
        {{ target.schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}

{%- endmacro %}
```

Elle dit à dbt : si un modèle déclare un schéma, prends-le tel quel ; sinon, prends celui du profil. C'est la version que tout projet finit par écrire, et celle que les trois projets de cette formation partagent. Elle ne changera plus jusqu'au capstone.

Notez la forme des commentaires : dans un fichier de Jinja pure comme une macro, un commentaire s'écrit `{# … #}` et non `--`, parce que dbt évalue ce qui ressemble à du Jinja même dans un commentaire SQL.

Étape 4 — Déclarer les sources

Théorie : `Theorie_03_AE.md`, section 4.

Dans `models/staging/_sources.yml` :

```
version: 2

sources:
  - name: raw
    description: "La couche RAW livrée par Data Engineering, 22 tables."
    schema: raw
    tables:
      - name: erp_kna1
        description: "Le référentiel des concessionnaires, un extrait par mois."
      - name: erp_vbak
        description: "Les en-têtes de commande."
```


© 2026 iAlexMG. Tous droits réservés.

Deux tables, et pas une de plus : ce sont celles que les modèles de ce labo vont lire. Déclarer une source qu'aucun modèle n'utilise ajoute une promesse que rien ne vérifie, et vous verrez la règle s'appliquer deux fois de suite. Les articles arrivent avec la question P1 des exercices, qui vous fera déclarer `erp_mara` ; les postes de commande arrivent au Cours 04, avec le fait des ventes.

Le schéma `argent`, qui porte les séries horaires de l'usine, se déclarera au Cours 06, quand l'étoile de l'usine le réclamera.

Étape 5 — Voir ce que dbt connaît

Théorie : `Theorie_03_AE.md`, sections 4 et 9.

```
dbt ls
```

Deux lignes sortent, une par source, préfixées de `source:` :

```
source:boreal_ae.raw.erp_kna1
source:boreal_ae.raw.erp_vbak
```

Aucun modèle encore : c'est normal, vous n'en avez pas écrit. dbt le signale d'ailleurs par un avertissement, « Configuration paths exist in your dbt_project.yml file which do not apply to any resources », qui vise le bloc `staging` que rien ne remplit. Il disparaîtra à l'étape 6.

Cette commande est celle qu'on lance quand un chiffre surprend, parce qu'elle dit exactement ce que dbt connaît, dans l'ordre où il le connaît. Retenez-la.

Étape 6 — La première vue de staging

Théorie : `Theorie_03_AE.md`, sections 6 et 8.

Dans `models/staging/stg_concessionnaires.sql` :

```
with courant as (

    select
        *,
        row_number() over (
            partition by kunnr
            order by _lot desc
        ) as rang

    from {{ source('raw', 'erp_kna1') }}
```


© 2026 iAlexMG. Tous droits réservés.

```
)

select
    kunnr as code_concessionnaire,
    name1 as nom_concessionnaire,
    ort01 as ville,
    regio as province,
    case kukla
        when '01' then 'Or'
        when '02' then 'Argent'
        when '03' then 'Bronze'
    end as palier,
    try_strptime(erdat, '%Y%m%d')::date as date_creation,
    loevm = 'X' as est_supprime,
    _lot as lot_source

from courant
where rang = 1
```

Les quatre gestes du staging y sont, et rien d'autre. Renommer, dans le `select`. Typer, par `try_strptime` et par la comparaison qui rend un booléen. Décoder, par le `case`. Tirer l'état courant, par la fonction de fenêtre et le `where rang = 1`.

Aucune jointure, et c'est la règle.

Construisez :

```
dbt run
```

Puis vérifiez dans la ligne de commande DuckDB, après avoir laissé dbt finir :

```
select
    count(*) as concessionnaires,
    count(*) filter (where palier is null) as paliers_vides,
    count(*) filter (where est_supprime) as fiches_fermees
from staging.stg_concessionnaires;
```

Attendu : 156 concessionnaires, 0 palier vide, 4 fiches fermées. La deuxième colonne est le contrôle du décodage, celui du Cours 02 : un `case` qui ne trouve rien rend `NULL` en silence.

Remarquez le nom du schéma : `staging`, et non `staging_staging`. C'est la macro de l'étape 3 qui l'a rendu nu.

© 2026 iAlexMG. Tous droits réservés.

Étape 7 — La vue qui retrouve les 25 483 commandes

Théorie : `Theorie_03_AE.md`, sections 6, 7 et 8.

Dans `models/staging/stg_commandes.sql` :

```
with courant as (

    select
        *,
        row_number() over (
            partition by vbeln
            order by _lot desc
        ) as rang

    from {{ source('raw', 'erp_vbak') }}

)

select
    vbeln as numero_commande,
    kunnr as code_concessionnaire,
    auart as type_document,
    try_strptime(erdat, '%Y%m%d')::date as date_creation,
    try_strptime(nullif(aedat, '00000000'), '%Y%m%d')::date as date_modification,
    netwr::decimal(18, 2) as valeur_nette,
    waerk as devise,
    _lot as lot_source

from courant
where rang = 1
```

Deux détails valent le détour. Le `nullif` traite la date vide de SAP comme une absence, sans quoi `try_strptime` rendrait `NULL` de toute façon, mais pour une mauvaise raison : on veut dire « pas de modification », pas « conversion ratée ». Et `netwr` se type en `decimal`, jamais en `double` : un montant se compte au cent près.

```
dbt run
```

Puis la vérification qui est le but du cours :

```
select
    (select count(*) from raw.erp_vbak) as lignes_dans_la_raw,
```


© 2026 iAlexMG. Tous droits réservés.

```
    (select count(*) from staging.stg_commandes) as commandes_dans_le_staging,
    (select count(*) from staging.stg_commandes where date_modification is not null) as commandes_modifiees;
```

Mesure	Attendu
Lignes dans la `RAW`	26 538
Commandes dans le `STAGING`	25 483
Commandes portant une date de modification	2 231

Les 1 055 lignes en trop de la `RAW` ont disparu, et aucune commande n'a été perdue. C'est le quatrième geste du staging, celui qu'on oublie, et vous venez de le faire tenir en six lignes de SQL.

Étape 8 — Relier deux modèles, et voir le graphe

Théorie : `Theorie_03_AE.md`, sections 7 et 9.

Un dernier modèle, qui ne sert à rien d'autre qu'à vérifier une promesse : toute commande doit pointer un concessionnaire connu.

Dans `models/staging/ctl_commandes_sans_concessionnaire.sql` :

```
select
    c.numero_commande,
    c.code_concessionnaire,
    c.date_creation

from {{ ref('stg_commandes') }} as c
left join {{ ref('stg_concessionnaires') }} as d
       on d.code_concessionnaire = c.code_concessionnaire

where d.code_concessionnaire is null
```

Remarquez que ce modèle joint, alors que la règle du staging l'interdit. C'est qu'il n'est pas un modèle de staging au sens de la couche : c'est un contrôle, et son préfixe `ctl_` le dit. Il ne sera consommé par personne.

```
dbt run
dbt ls
```

`dbt ls` liste maintenant trois modèles et deux sources. L'ordre d'exécution vous l'a déjà montré : dbt a construit les deux vues de staging avant le contrôle, parce que `ref()` lui a dit qu'il en dépendait. Vous ne le lui avez dit nulle part ailleurs.

© 2026 iAlexMG. Tous droits réservés.

La vérification :

```
select count(*) as commandes_sans_concessionnaire
from staging.ctl_commandes_sans_concessionnaire;
```

Attendu : 0.

Ce zéro mérite qu'on s'y arrête, parce qu'il est le produit d'un choix. Un concessionnaire de Boréal a passé une commande avant que sa fiche n'arrive : la commande est livrée au lot de mai 2024, sa première fiche au lot de juin. À l'époque, cette commande était orpheline. En tirant l'état courant, vous avez regardé le référentiel d'aujourd'hui, où la fiche existe, et le problème a disparu.

Vérifiez-le par vous-même :

```
select
    (select count(*) from raw.erp_kna1 where kunnr = '0000200154' and _lot = '2024-05') as fiche_en_mai,
    (select min(_lot) from raw.erp_kna1 where kunnr = '0000200154') as premiere_fiche,
    (select _lot from raw.erp_vbak where vbeln = '0010015405') as lot_de_la_commande;
```

Attendu : 0 fiche en mai, première fiche au lot `2024-06`, commande livrée au lot `2024-05`.

Au Cours 05, quand la dimension passera en type 2 et que le fait pointera la version d'époque, la même question donnera une autre réponse. Les deux seront justes, et il faudra savoir laquelle on pose.

Récapitulatif

L'état du projet à la fin du labo :

Ce qui est en place	**Vérification**
Le projet dbt dans `02_ae`	`dbt debug` rend « All checks passed! »
Les schémas nus	les vues vivent dans `staging`, pas `staging_staging`
Deux sources déclarées	`dbt ls` les liste, préfixées de `source:`
Trois modèles	`dbt run` rend `PASS=3 WARN=0 ERROR=0`
Le staging des concessionnaires	156 lignes, 0 palier vide, 4 fiches fermées
Le staging des commandes	25 483 lignes, contre 26 538 dans la source

© 2026 iAlexMG. Tous droits réservés.

Ce qui est en place	Vérification
Le contrôle	0 commande sans concessionnaire

Le Cours 04 ouvre la couche des `MARTS` et bâtit la première étoile, celle que le Cours 02 a conçue. Le prototype que vous aviez écrit à la main y renaîtra, mais versionné, testé et rejouable.

Pensez à commiter : c'est la première fois que votre dépôt porte du travail.

```
git add dbt_project.yml macros models
git commit -m "Cours 03 : le projet dbt et la couche staging"
```

Dépannage

Le message	Ce qu'il veut dire	La parade
`cannot access local variable 'summary_message'`	`profiles.yml` existe mais est vide	le remplir, ou le supprimer pour obtenir un vrai message
`profiles.yml file [ERROR invalid]`, puis « Connection test skipped since no profile was found »	le nom du profil diffère entre les deux fichiers	comparer la première ligne de `profiles.yml` et la clé `profile` du projet ; le message ne nomme ni l'un ni l'autre
`IO Error: Cannot open file ... Le chemin d'accès spécifié est introuvable`	chemin relatif dans `profiles.yml`, ou faute de frappe	écrire un chemin absolu, avec des barres obliques
`IO Error: ... utilisé par un autre processus`	la ligne de commande DuckDB est restée ouverte	la fermer par `.exit` avant de lancer dbt
Les vues atterrissent dans `staging_staging`	la macro `generate_schema_name` manque ou n'est pas dans `macros/`	vérifier l'étape 3, puis relancer `dbt run`
`Compilation Error ... depends on a source named 'raw.xxx' which was not found`	un modèle lit une source non déclarée	l'ajouter à `_sources.yml`
`dbt init` demande un nom alors qu'on lui en a donné un	le nom commence par un chiffre	ce labo n'utilise pas `dbt init` ; écrire `dbt_project.yml` à la main

---

© 2026 iAlexMG. Tous droits réservés.

Annexe — dbt sur Snowflake et sur Databricks

Écrite d'après la documentation de dbt, de Snowflake et de Databricks, consultée le 2026-09-23. Rien de cette annexe n'a été exécuté : aucun compte n'a servi à l'écrire, et aucun identifiant n'y figure. Elle suppose la `RAW` chargée sur la plateforme par l'annexe du Labo 03 de Data Engineering.

Le projet ne change pas d'une ligne : un modèle reste un `select`, une source se déclare, `ref()` fait le graphe. Ce qui change tient à trois choses, l'adaptateur, la connexion et ce qu'on paie (section 10 de la théorie), et, sur Snowflake, à un mot qui prête à confusion.

Sur Snowflake

L'adaptateur s'installe à côté de dbt-core, dans l'environnement de la formation, par `pip install dbt-snowflake`.

dbt travaille sous un rôle à lui, pas sous `SYSADMIN`. Data Engineering a créé la base `BOREAL` et le schéma `RAW` sous `SYSADMIN`, le rôle prévu pour posséder les objets de données, et c'est pour cela que ce rôle peut aussi tout détruire. dbt n'a besoin que de trois droits : se servir d'un entrepôt virtuel, lire la `RAW` et créer ses propres schémas. Lui prêter `SYSADMIN`, c'est lui permettre d'effacer la `RAW` sur une faute de frappe dans une macro. La documentation de dbt nomme ce rôle `transformer` et son entrepôt `transforming`. Un administrateur le crée une fois :

```
use role securityadmin;
create role if not exists transformer;
grant role transformer to user <votre_utilisateur>;

use role sysadmin;
create warehouse if not exists transforming
    warehouse_size = xsmall auto_suspend = 60 auto_resume = true;
grant usage on warehouse transforming to role transformer;
grant usage, create schema on database boreal to role transformer;
grant usage on schema boreal.raw to role transformer;
grant select on all tables in schema boreal.raw to role transformer;
grant select on future tables in schema boreal.raw to role transformer;
```

Les rôles, leur hiérarchie et la propriété des objets appartiennent au Cours 03 de la formation Snowflake ; il suffit ici de savoir que le rôle qui crée un objet en devient propriétaire, et que `transformer` possédera donc les schémas que dbt crée, et rien d'autre. L'entrepôt s'éteint après 60 secondes sans requête, et c'est ce qui fixe la facture : Snowflake facture le temps où un entrepôt tourne, pas le nombre de modèles.

Le profil. D'après la documentation de dbt, l'authentification par mot de passe cesse d'être acceptée le 31 août 2026, au profit de la paire de clés ou d'OAuth. Avec une paire de clés, la clé privée et sa phrase de passe restent dans des variables d'environnement de votre poste :

```
boreal_ae:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: <identifiant_du_compte>
```


© 2026 iAlexMG. Tous droits réservés.

```
    user: <votre_utilisateur>
    private_key: "{{ env_var('SNOWFLAKE_PRIVATE_KEY') }}"
    private_key_passphrase: "{{ env_var('SNOWFLAKE_PRIVATE_KEY_PASSPHRASE') }}"
    role: transformer
    warehouse: transforming
    database: boreal
    schema: staging
    threads: 4
```

C'est ici que la règle du `.gitignore` compte pour de bon : ce fichier nomme un compte, un utilisateur et les variables qui portent le secret. Les sources du projet n'ont rien à changer, `raw` étant un schéma de la base `boreal` que le profil désigne.

Un mot, une fois pour toutes : *transient*. Snowflake protège une table ordinaire, dite permanente, de deux façons, le voyage dans le temps, qui garde les versions passées, puis sept jours de sécurité intégrée (*Fail-safe*), réservés à la reprise après sinistre. Une table transient n'a pas de sécurité intégrée, et son voyage dans le temps dure au plus un jour.

	Permanente	**Transient**
Voyage dans le temps	0 ou 1 jour, jusqu'à 90 jours en édition Enterprise ou supérieure	0 ou 1 jour
Sécurité intégrée	7 jours	aucune
Ce qu'elle coûte	le stockage des versions gardées	moins
Pour quoi	ce qui ne se refait pas	ce qui se refait

La dernière ligne est la règle, et elle répond à la question que pose l'annexe de Data Engineering, où la `RAW` vit dans un schéma transient : la `RAW` se recharge à partir des fichiers livrés, qui sont sa vraie copie de sûreté, et un schéma transient rend transient tout ce qu'on y crée. Du côté de dbt, la documentation dit que toutes les tables que dbt crée sur Snowflake sont transient par défaut. C'est juste pour les tables de `MARTS`, qui se refont par `dbt build` à partir de la `RAW`, et sans objet pour les vues de `STAGING`, qui ne stockent rien. Ce ne l'est pas pour le snapshot du Cours 05, le seul objet du projet qui accumule une histoire que la source pourrait ne pas garder : le Cours 09 le mesure, et le snapshot se déclare permanent, dans `dbt_project.yml` :

```
  snapshots:
    boreal_ae:
      +transient: false
```

La page de dbt ne montre cette clé que sur des modèles. Vérifiez donc sur le compte ce que le snapshot est devenu : `show tables like 'SNP_CONCESSIONNAIRE' in schema boreal.snapshots;` rend une colonne `kind`, qui vaut `TABLE` pour une table permanente et `TRANSIENT` sinon.

© 2026 iAlexMG. Tous droits réservés.

Le reste du vocabulaire qui prête à confusion, la section 10 de la théorie le règle : le schéma `STAGING` de votre projet est une couche de modélisation ; un stage est l'endroit où Data Engineering dépose les fichiers avant `COPY INTO`, parce que les entrepôts de Snowflake ne voient pas le disque de votre poste. dbt n'y touche jamais.

Sur Databricks

L'adaptateur, `pip install dbt-databricks`. La connexion passe par un entrepôt SQL et Unity Catalog, et la documentation de Databricks recommande OAuth : à la première commande, dbt ouvre le navigateur pour vous identifier, et aucun secret n'entre dans le fichier. La documentation de dbt présente le jeton d'accès personnel comme un mode hérité.

```yaml
boreal_ae:
  target: dev
  outputs:
    dev:
      type: databricks
      host: <votre_espace>.cloud.databricks.com
      http_path: /sql/1.0/warehouses/<identifiant_de_l_entrepot>
      catalog: workspace
      schema: staging
      auth_type: oauth
      threads: 4
```

Le catalogue `workspace` est celui que l'annexe de Data Engineering et le kit 02 emploient par défaut, et la `RAW` y vit dans le schéma `raw`. Ni transient ni sécurité intégrée chez Databricks : une table Delta garde l'historique de ses versions, et le Cours 10 de Data Engineering enseigne le voyage dans le temps qui en découle. On y paie le temps de l'entrepôt SQL, que l'édition gratuite fournit et facture à l'usage.

© 2026 iAlexMG. Tous droits réservés.